Week 9
Proximal Policy Optimization (Deep Dive)

STUDY-RL — a self-paced Deep RL curriculum

October 20, 2026

Contents

Learning objectives 3

1 Motivation and intuition 4

2 Mathematical formulation 5
- 2.1 Importance sampling and the surrogate . . . 5
- 2.2 PPO-clip . . . 5
- 2.3 PPO-KL (adaptive penalty) . . . 5
- 2.4 The full PPO loss . . . 6

3 Algorithms 7
- 3.1 The 37 implementation details . . . 7

4 Implementation walkthrough 9
- 4.1 Vector env + GAE collection (details 1, 5, 6, 12) . . . 9
- 4.2 PPO loss (details 7, 8, 9, 11) . . . 9
- 4.3 Linear LR schedule + Adam $\varepsilon = 10^{-5}$ (details 3, 4) . . . 9

5 Empirical results 11

6 Safety angle 13

7 Matching application 14

8 Exercises 15

9 Further reading 16

Learning objectives

Study companion. Read `teaching_guide.md` in this folder for step-by-step examples, an analogy with its limits, and self-checks. Then use `workbook.ipynb` to implement and vary the calculations.

Key Idea

By the end of this week you will be able to:

1. Motivate PPO's clipped surrogate using TRPO's trust-region idea; explain why clipping is a heuristic objective rather than a hard trust-region constraint.
2. State and implement the PPO-KL variant (adaptive penalty) and contrast its behaviour with PPO-clip.
3. Audit the task-relevant implementation choices using the source article and the course checklist; verify each enabled option in code.
4. Train PPO on BipedalWalker-v3 with vector envs and reproduce the canonical learning curve.
5. Understand why PPO is the universal workhorse of RLHF and what about its design makes it the right algorithm for LLM fine-tuning (preview of week 18).

1 Motivation and intuition

TRPO works but is expensive: every update requires a conjugate-gradient solve of $Fx = g$ ($\sim$10 Fisher–vector products), a back-tracking line search, and a separate value-function fit. For deep RL at scale, this is a non-starter. PPO is the engineer's compromise: keep the trust-region *idea* (don't move the policy too far per update) but replace the second-order solve with a first-order surrogate that mini-batch SGD can handle, and run multiple epochs of mini-batch updates per rollout.

Engstrom et al. (2020) demonstrate implementation sensitivity in controlled comparisons; they do not establish a universal 20–80 split.

Intuition. Picture the importance ratio $\rho_t(\theta) = \pi_\theta(a_t \mid s_t)/\pi_{\theta_{\text{old}}}(a_t \mid s_t)$. An exact action importance ratio still changes measure correctly at a fixed state, but large ratios increase variance and the old-policy state distribution becomes stale. PPO compares the unclipped and clipped objectives and removes an incentive for further improvement beyond a threshold. It does not constrain every actual ratio to $[1-\varepsilon, 1+\varepsilon]$ or guarantee a KL bound.

PPO as the universal workhorse

PPO is used in continuous-control benchmarks, multi-agent variants such as MAPPO, and preference training such as InstructGPT. Its first-order update is convenient, but its performance depends on the implementation, task, and hyperparameters. Neither clipping nor a commonly used recipe guarantees robustness or a particular scaling law.

Common pitfall

"It worked in their paper, why doesn't it work for me?" is almost always one of the 37 details. The two most common: forgetting advantage normalisation (the gradient ends up dominated by a few high-magnitude samples), and forgetting that value clipping needs the *old* value estimate, not the current one.

2 Mathematical formulation

2.1 Importance sampling and the surrogate

We collect rollouts under $\pi_{\theta_{\text{old}}}$, then optimise the importance-sampled objective

$$L^{\text{IS}}(\theta) = \mathbb{E}_{(s,a)\sim\pi_{\theta_{\text{old}}}}\left[\frac{\pi_\theta(a \mid s)}{\pi_{\theta_{\text{old}}}(a \mid s)}\hat{A}(s,a)\right] = \mathbb{E}[\rho(\theta)\hat{A}].$$

Why this is the right surrogate. Differentiating in θ at $\theta = \theta_{\text{old}}$:

$$\nabla_\theta L^{\text{IS}}\big|_{\theta_{\text{old}}} = \mathbb{E}\left[\frac{\nabla_\theta \pi_\theta(a \mid s)}{\pi_{\theta_{\text{old}}}(a \mid s)}\Big|_{\theta_{\text{old}}}\hat{A}\right] = \mathbb{E}\Big[\nabla_\theta \log \pi_\theta(a \mid s)\big|_{\theta_{\text{old}}}\hat{A}\Big],$$

which is exactly the policy gradient with advantage $\hat{A}$. With an appropriate advantage estimator, the surrogate matches the policy-gradient direction at θ_{old}. Away from that point it still uses old-policy states and advantages; it is not the exact new-policy return. Clipping discourages large updates without proving a bound on this mismatch.

2.2 PPO-clip

Definition 2.2.1 (PPO clipped surrogate)**.** With $\rho_t(\theta) \triangleq \pi_\theta(a_t \mid s_t)/\pi_{\theta_{\text{old}}}(a_t \mid s_t)$,

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t\Big[\min\Big(\rho_t(\theta)\hat{A}_t,\ \text{clip}(\rho_t(\theta), 1-\varepsilon, 1+\varepsilon)\hat{A}_t\Big)\Big].$$

Per-sample analysis. Fix one (s_t, a_t) with advantage $\hat{A}_t$. Two cases:

- $\hat{A}_t > 0$ (this action looked good): the surrogate is $\rho_t\hat{A}_t$ until $\rho_t = 1+\varepsilon$, where it saturates. Above $1+\varepsilon$ the gradient is zero — "stop pushing this action's probability higher; we've already increased it enough".
- $\hat{A}_t < 0$ (this action looked bad): the surrogate is $\rho_t\hat{A}_t$ until $\rho_t = 1-\varepsilon$, where it saturates. Below $1-\varepsilon$ the gradient is zero — "stop pushing this action's probability lower; it's already small enough".

The clip is asymmetric in $\hat{A}$. The min makes it a *pessimistic* bound on the surrogate: we never let optimism beyond the clip range drive the update.

2.3 PPO-KL (adaptive penalty)

The original PPO paper also presents

$$L^{\text{KL}}(\theta) = \mathbb{E}_t\Big[\rho_t(\theta)\hat{A}_t\Big] - \beta\,\mathbb{E}_t\big[\text{KL}\big(\pi_{\theta_{\text{old}}}(\cdot \mid s_t) \,\|\, \pi_\theta(\cdot \mid s_t)\big)\big],$$

with β adapted online: if measured KL exceeds the target by $1.5\times$, double β; if it falls under the target by $0.67\times$, halve it. This is the closest first-order approximation to TRPO's hard KL constraint.

Empirical verdict. Clip wins in practice. KL-penalty is slightly more theoretically clean (Pinsker-based safety angle) but tunes worse out of the box. The exception: RLHF, where the KL term is kept anyway (as a regulariser against reward hacking) and clip is layered *on top* of it.

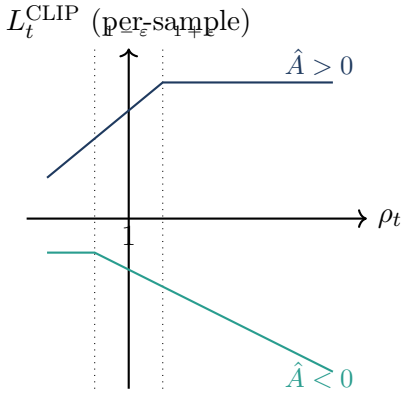


Figure 2.1: PPO-clip per-sample surrogate as a function of the importance ratio ρ_t. The blue (positive advantage) branch flattens above $1+\varepsilon$; the teal (negative advantage) branch flattens below $1-\varepsilon$.

2.4 The full PPO loss

$$L^{\mathrm{PPO}}(\theta,\phi) = \underbrace{L^{\mathrm{CLIP}}(\theta)}_{\text{policy}} - c_v \underbrace{\mathbb{E}_t\Big[(V_\phi(s_t) - \hat{R}_t)^2\Big]}_{\text{value}} + c_H \underbrace{\mathbb{E}_t[\mathcal{H}(\pi_\theta(\cdot \mid s_t))]}_{\text{entropy bonus}}.$$

With value clipping enabled (course detail #8), the value loss becomes

$$L^{\mathrm{V}}(\phi) = \mathbb{E}_t\Big[\max\Big((V_\phi(s_t) - \hat{R}_t)^2,\ (\mathrm{clip}(V_\phi(s_t), V_{\mathrm{old}} - \varepsilon, V_{\mathrm{old}} + \varepsilon) - \hat{R}_t)^2\Big)\Big].$$

3 Algorithms

PPO (Schulman 2017) — the canonical loop

Hyper-params: N vec envs, rollout $T_R = 2048$, mini-batches $M = 64$, epochs $K = 10$, $\gamma = 0.99$, $\lambda = 0.95$, $\varepsilon = 0.2$, $c_v = 0.5$, $c_H = 0.01$, Adam lr $= 3 \times 10^{-4}$ (annealed linearly), grad-clip 0.5.
Repeat:

1. Collect $N \times T_R$ transitions under $\pi_{\theta_{\text{old}}}$, store $(s, a, r, d, \log \pi_t^{\text{old}}, V_t^{\text{old}})$.
2. Bootstrap value for the final state of each env; compute GAE $\hat{A}_t$ and returns $\hat{R}_t = \hat{A}_t + V_t^{\text{old}}$.
3. Standardise $\hat{A}$.
4. **For** $k = 1, \ldots, K$ **epochs:**
   a) Shuffle indices; iterate over mini-batches of size M.
   b) Compute ρ_t, L^{CLIP}, value loss (clipped), entropy bonus; sum into L^{PPO}.
   c) Optionally early-stop if approximate KL $\geq 1.5 \cdot \text{KL}_{\text{target}}$.
   d) Optimise one Adam step with grad-clip.
5. $\theta_{\text{old}} \leftarrow \theta$, $V_{\text{old}} \leftarrow V_\phi$.

Complexity per update: $\mathcal{O}(NT_RK)$ in transitions; backbone of all modern on-policy RL.

3.1 The 37 implementation details

The canonical list (Huang et al., *The 37 Implementation Details of Proximal Policy Optimization*, ICLR Blog 2022 & `iclr-blog-track.github.io/2022/03/25/ppo-implementation-details/`) groups details into core, Atari-specific, continuous-control-specific, LSTM, and multi-discrete variants. We list and motivate them.

#	Detail	Category
1	Vectorised env (Sync or Async) for parallel rollouts.	core
2	Orthogonal weight init: actor final layer std 0.01, value head std 1.0, hidden std $\sqrt{2}$.	core
3	Adam, $\varepsilon = 10^{-5}$ (not the default 10^{-8}).	core
4	Linear learning-rate schedule (decay to zero).	core
5	GAE with $\gamma = 0.99, \lambda = 0.95$.	core
6	Per-mini-batch advantage normalisation.	core
7	PPO-clip $\varepsilon = 0.2$; log-ratio bounds require $\log(1 \pm \varepsilon)$.	core
8	Value-function clipping (clip range ε).	core
9	Joint policy+value MSE+entropy loss, weights $1, 0.5, 0.01$.	core
10	Global gradient norm clipping at 0.5.	core
11	Approximate-KL early stopping at $1.5 \cdot \mathrm{KL}_{\mathrm{target}}$.	core
12	Independent observation/return normalisation (running mean/std).	core
13	Returns clipping (clip to ± 10) for stability with value clipping.	core
14	Reward clipping to ± 10.	core
15	State-independent log-std (continuous).	continuous
16	tanh activation in the policy (vs. ReLU in value).	continuous
17	Initial log-std 0 (not -0.5).	continuous
18	Action clipping to $[-1, 1]$ after sampling.	continuous
19	Frame skip 4 (Atari) / no skip (continuous).	Atari
20	Frame stack 4 (Atari).	Atari
21	Greyscale + 84×84 resize (Atari).	Atari
22	"Fire-on-reset" action (Atari).	Atari
23	Episodic life wrappers (Atari).	Atari
24	Reward sign clipping (Atari only).	Atari
25	NoOp reset 0–30 steps (Atari).	Atari
26	MaxAndSkipEnv for frame skip (Atari).	Atari
27	Conv-trunk init: orthogonal $\sqrt{2}$.	Atari
28	Final policy head init: orthogonal 0.01.	core
29	Final value head init: orthogonal 1.0.	core
30	Mini-batch size $\leq$ rollout / 4 (typically).	core
31	$K = 10$ epochs (continuous), $K = 4$ (Atari).	core
32	Seed each RNG reproducibly; use distinct seeds for vector environments.	core
33	Deterministic CUDNN *off* for speed.	core
34	LSTM: `burn-in` hidden state from rollout for first T_b steps.	LSTM
35	LSTM: zero-out hidden state at episode boundaries.	LSTM
36	Reward-to-go normalisation (alt to advantage normalisation).	alt
37	Mixed-precision off by default (FP32 matters for small policies).	core

Table 3.1: Course audit prompts inspired by Huang et al. (2022), not a verbatim or exhaustive implementation of their numbered list. Some rows are task-specific alternatives; inspect the current trainer before marking one implemented.

4 Implementation walkthrough

The trainer retains the sampled Gaussian latent action for its likelihood; only the action sent to the environment is clipped to the action-space bounds. Scoring a clipped action as the original draw would invalidate likelihood ratios. A true terminal removes the value bootstrap, while a time-limit truncation bootstraps from the final observation and stops the GAE trace before the reset episode. The common rollout helpers request same-step autoreset and recover the final observation supplied in its info dictionary.

4.1 Vector env + GAE collection (details 1, 5, 6, 12)

```
# Use the version-aware helper used by project/train.py.
from common.rollouts import sync_vector_env
# Inspect its arguments before constructing environments. The helper
# requests same-step autoreset and preserves final observations.
# Store sampled Gaussian actions for log probabilities; only clip the
# copy sent to the environment.
```

4.2 PPO loss (details 7, 8, 9, 11)

```
def ppo_loss(logp, logp_old, value, value_old, advantages, returns,
             clip_eps=0.2, value_coef=0.5, entropy_coef=0.01,
             entropy=None):
    ratio = torch.exp(logp - logp_old)
    surr1 = ratio * advantages
    surr2 = torch.clamp(ratio, 1 - clip_eps, 1 + clip_eps) * advantages
    pi_loss = -torch.min(surr1, surr2).mean()

    v_clipped = value_old + torch.clamp(value - value_old, -clip_eps, clip_eps)
    v_loss1 = (value - returns) ** 2
    v_loss2 = (v_clipped - returns) ** 2
    v_loss = 0.5 * torch.max(v_loss1, v_loss2).mean()

    H = entropy.mean() if entropy is not None else torch.tensor(0.0)
    loss = pi_loss + value_coef * v_loss - entropy_coef * H

    approx_kl = (logp_old - logp).mean()
    return loss, pi_loss, v_loss, H, approx_kl
```

4.3 Linear LR schedule + Adam $\varepsilon = 10^{-5}$ (details 3, 4)

```
opt = torch.optim.Adam(net.parameters(), lr=3e-4, eps=1e-5)
for it in range(n_iters):
    frac = 1.0 - it / n_iters
    for g in opt.param_groups:
        g["lr"] = 3e-4 * frac
```

5 Empirical results

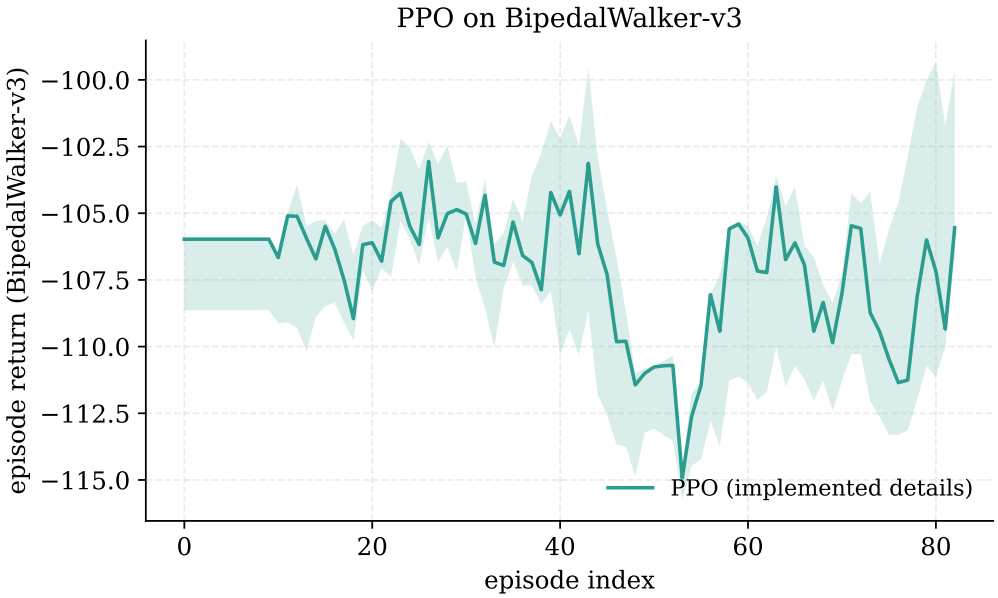


Figure 5.1: PPO on BipedalWalker-v3. Mean and IQR over 3 seeds, $N = 8$ vec envs, $T_R = 512$, $K = 4$ epochs, $\varepsilon = 0.2$.

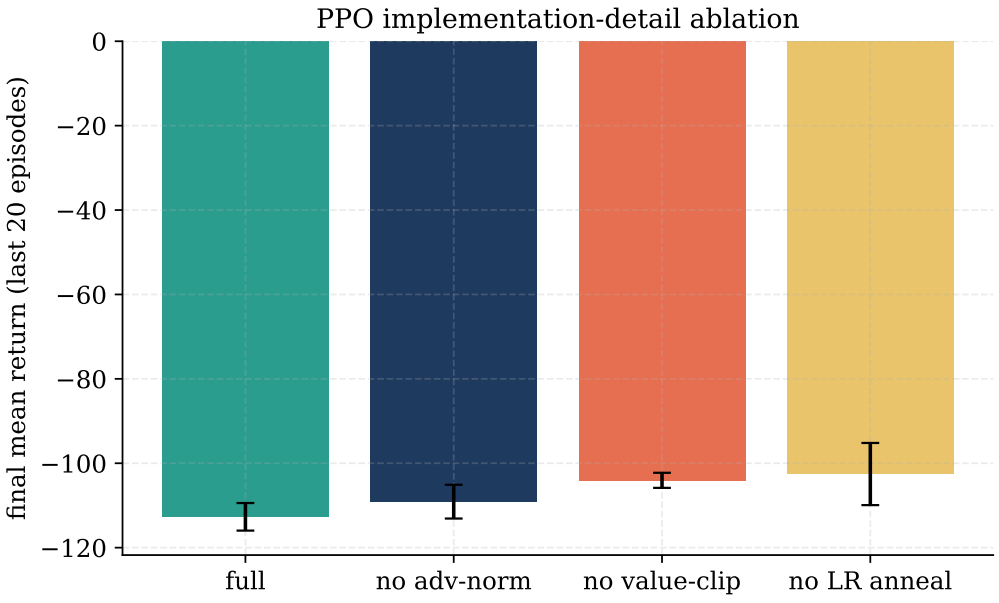


Figure 5.2: The cost of skipping details: PPO with vs. without advantage normalisation (#6), value clipping (#8), and learning-rate annealing (#4). Effects and ordering need not transfer to another task or budget.

6 Safety angle

Safety angle

PPO is the algorithm of RLHF. Stiennon et al. 2020 and Ouyang et al. 2022 both use PPO as the RL backbone. The KL penalty between π_θ and the SFT reference π_{ref} is added to the reward signal:

$$r_{\text{shaped}}(s,a) = r_{\text{RM}}(s,a) - \beta \log \frac{\pi_\theta(a \mid s)}{\pi_{\text{ref}}(a \mid s)}.$$

Two different anchors. PPO clipping discourages excessive improvement on sampled actions relative to the rollout policy. It does not bound per-step or total KL: shared parameters and other samples can move an already-clipped ratio further. The KL-to-SFT-reference reward penalty serves a separate purpose and a different reference distribution. Monitor both divergences; neither a finite penalty nor clipping alone is a hard safety constraint.

Failure modes (concrete).

1. **Reward over-optimisation.** Gao–Schulman–Hilton (2022) fit gold-model performance in a synthetic reward-model setup, with different forms for RL and best-of-n. Their result is not a universal quadratic law for the proxy–true reward gap. Mitigation: KL-penalty (above), early stopping by KL.
2. **Mode collapse / repetition.** Reward optimisation can reduce output diversity. Monitor diversity and held-out quality; entropy and reference-KL penalties have different roles, and no universal entropy coefficient prevents collapse.
3. **Distribution shift in the value head.** The value net is trained on π_{old} rollouts; if π moves too fast, V_ϕ is stale and advantages are biased. Mitigation: lower learning rate, more epochs to fit V relative to π.
4. **Spec-gaming by clip exploitation.** If a state has hugely positive advantage and ρ_t jumps above $1+\varepsilon$ before the gradient saturates, you can still get a destructive update. Mitigation: detail #11 (KL early stopping).

A preview of week 18: every one of these failure modes will reappear when we train PPO on a Pythia-160M with LoRA, and the mitigations are identical.

7 Matching application

Matching application

PPO as the slate-ranking optimiser for synthetic ATS / synthetic insurance. The leap from REINFORCE-for-ranking (week 7) to PPO-for-ranking is modest: replace the single-update gradient step with the PPO-clip surrogate over a mini-batch of logged slates, run for $K = 4$ epochs, and apply value-function clipping. The benefits:

- **Stability under reward-model drift.** A click-through-rate proxy reward r_{RM} that gets re-trained nightly cannot induce a runaway optimiser; PPO-clip discourages large updates, while measured KL and independent evaluation remain necessary.
- **Mini-batch parallelism.** The same logged batch supports multiple SGD steps; four passes make four optimisation passes over the same data, not four independent samples or a guaranteed fourfold information gain.
- **Direct KL audit.** The approximate-KL metric is logged at every update; an alarm at KL > 0.05 catches a misbehaving update before it ships.

Concrete recipe for synthetic insurance weekly retrains.

1. Collect last 7 days of logged slates with behaviour-policy probabilities.
2. Check logger coverage and propensity accuracy; SNIPS estimates policy value, not a canonical per-transition advantage. Reusing historical logs requires a separately justified off-policy learning objective (week 11).
3. PPO-clip update: 4 epochs, $\varepsilon = 0.1$ (tighter than the standard 0.2 because logged data has more importance-ratio variance), early-stop on KL$> 10^{-2}$.
4. Online A/B with the new θ for 48 hours; gate the rollout on a CTR-improvement test and a slot-share-stability check.

This is an application sketch, not an established universal recipe: PPO normally refreshes data from its current rollout policy, and stale logged slates require additional off-policy reasoning.

8 Exercises

E1 ($\star$) Implement `ppo_clip_loss(ratio, adv, eps)` and unit-test edge cases.

E2 ($\star$) Show that L^{CLIP} is differentiable everywhere except $\rho \in \{1-\varepsilon, 1+\varepsilon\}$ and explain why this is fine in practice.

E3 ($\star\star$) Implement value-function clipping; compare enabled and disabled variants on a 50-update toy without assuming the result.

E4 ($\star\star$) Train PPO on BipedalWalker-v3; reach mean return ≥ 100 within 800 updates.

E5 ($\star\star$) Ablate three details from the 37; produce Figure 5.2.

E6 ($\star\star$) Compute the empirical KL between π_θ and $\pi_{\theta_{\text{old}}}$ each update; verify the KL early-stop fires sensibly.

E7 ($\star\star\star$) Implement PPO-KL (adaptive penalty) and compare to PPO-clip head-to-head.

E8 ($\star\star\star$, meaningful objective) Add a $-\beta \text{KL}(\pi \| \pi_{\text{ref}})$ term to the reward (RLHF-style); measure KL for several positive penalties; explain why a finite penalty does not enforce a specified KL ball when rewards change scale.

9 Further reading

See `extended_reading.pdf`. Five papers: Schulman et al. 2017 (PPO), Huang et al. 2022 (37 details blog & ICLR poster), Engstrom et al. 2020 (Implementation Matters), Andrychowicz et al. 2021 (*What Matters in On-Policy RL?*), and Ouyang et al. 2022 (InstructGPT — PPO for RLHF). Sourced from `research/deeprl_core_sota.md` §2.6 and `research/llm_rl_sota.md`.